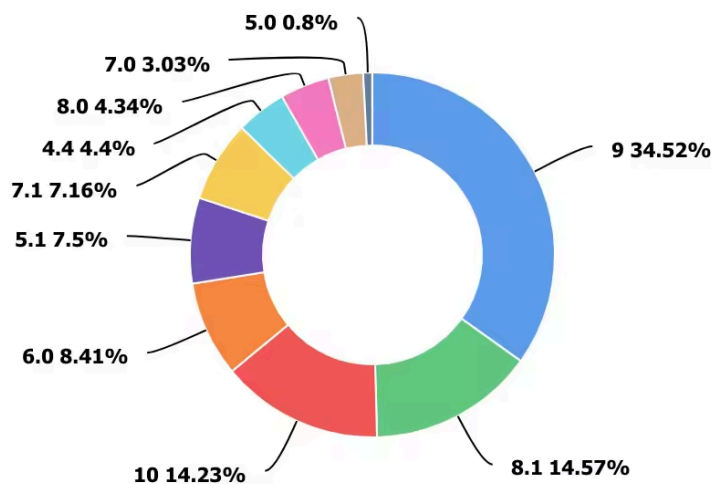


AMAP跨端技术演变

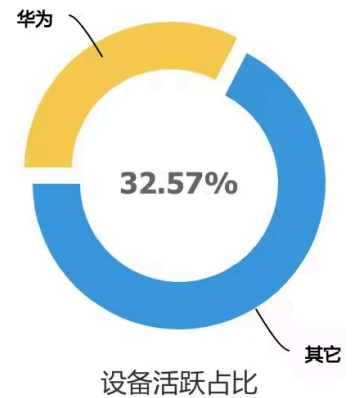
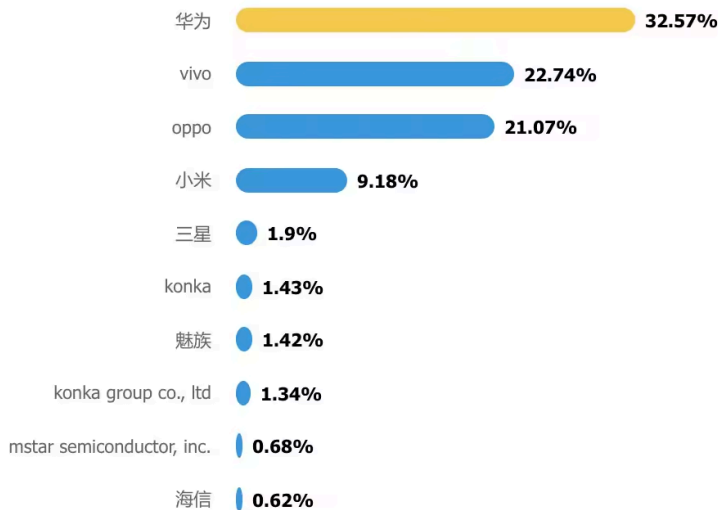
跨端背景

移动端按系统划分主要为Android和iOS，2019年安卓手机占87%市场份额，iPhone占13%，从以下数据可以看出我们要实现一个移动端产品，要实现两个平台10个版本，Android需要考虑7个版本的兼容问题，iOS至少考虑3个版本的兼容问题，Android除了官方版本之外，还有大量第三方厂商定制版，兼容它们是一个庞大的工作量，如果能实现一处开发多端通用的跨端框架，将会为公司节省大量的研发成本，同时提升产品的研发效率（不需要考虑多版本的兼容问题），在用户体验上也能保持高度统一。

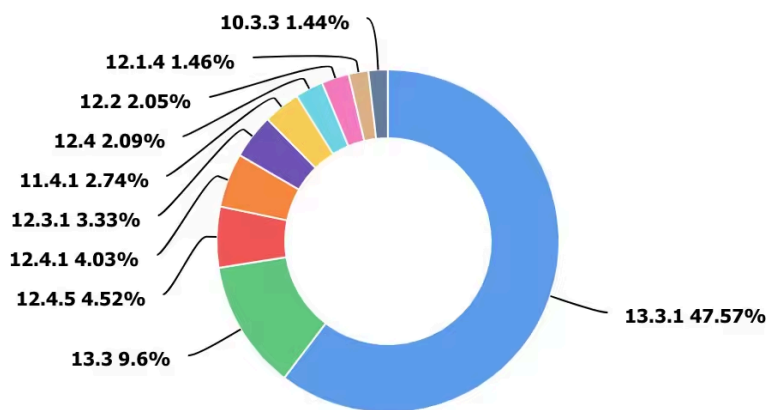
Android 系统分布



版本	占比
9	34.52%
8.1	14.57%
10	14.23%
6.0	8.41%
5.1	7.5%
7.1	7.16%
4.4	4.4%



iOS 系统分布



版本	占比
13.3.1	47.57%
13.3	9.6%
12.4.5	4.52%
12.4.1	4.03%
12.3.1	3.33%
11.4.1	2.74%
12.4	2.09%

移动端跨端现状

目前移动端跨端框架百花齐放，颇有一股推倒原生开发者的势头，如cordova、ionic、react-native、weex、kotlin-native、flutter等。

移动端跨端开发，大致归纳为以下几种情况：

- `react native`、`weex` 均使用JavaScript作为编程语言，目前JavaScript在跨平台开发中，可谓占据半壁江山，大有“一统天下”的趋势。实现原理：Javascript 代码中所生成的dom，最后都是由 Native 端解析，再得到对应的Native控件渲染，如 Android 中 `<text>` 标签对应 `WXTextView` 控件。
- `kotlin-native` 开始支持 iOS 和 Web 开发，（kotlin已经成为android的一级语言）也想尝试“一统天下”。实现原理：将Kotlin源码编译成不需要任何VM支持的目标平台二进制数据的技术，编译后的二进制数据可以直接运行在目标平台上，它主要包含一个基于LLVM的后端编译器和一个Kotlin本地运行时库。

- `flutter` 是Google跨平台移动UI框架，使用dart作为编程语言，目前社区也很火热，国内几个大厂都在尝试使用。实现原理：Flutter 不使用native原生控件，而是使用自己的Engine 来绘制 Widget（Flutter的显示单元），而 Dart 代码都是通过 AOT 编译为平台的原生代码，所以 Flutter 可以直接与平台通信，不需要JS引擎的桥接



AMAP跨端演变

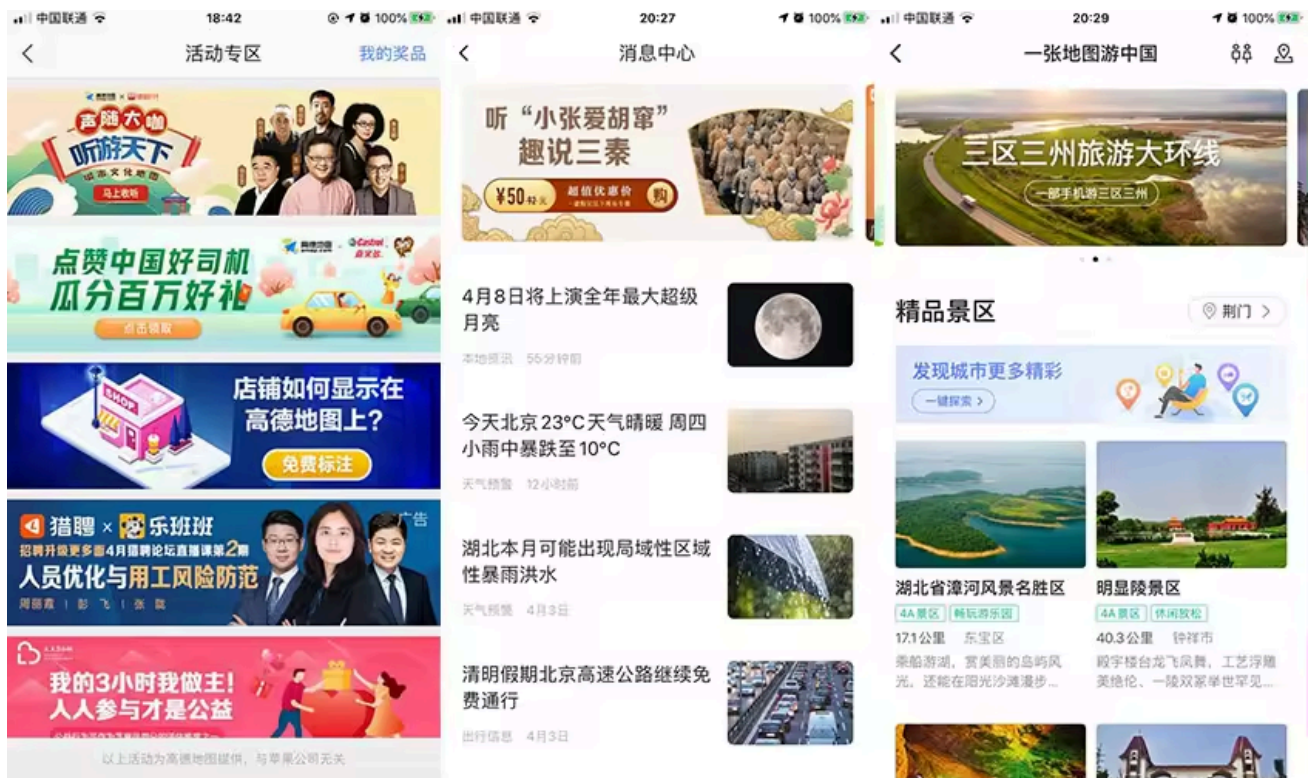
AMAP以地图为核心，支撑一系列与地图相关的业务如导航、打车、POI等，对外提供两个平台的客户端，因此相同业务需要不同平台的研发进行开发，提测、发布等一系列流程都是各平台一份，这样会造成很大的人力浪费，开发一个功能最少需要两个人，QA测试也需要双倍人力投入，产品体验上也不能完全保持统一（平台特性不一致），技术上代码也无法复用，因此对跨端框架有很强烈的需求。AMAP跨端实现是根据自身业务一步步演变过来的，在演变同时也借鉴了一些行业经验。AMAP跨端技术为动态UI，代号为AJX。

WebView JSBridge

最初AMAP仅有的跨端方案为WebView + JSBridge这种通用且成本最低的方案，也是业界最常用的跨端方案，这种方案是利用浏览器的跨平台特性实现的，缺点：

- WebView的渲染效率和JS执行性能不佳，跟原生体验相差太远
- Android 手机系统版本和设备厂商的定制，很难保证所在所有设备上都能提供一致的体验

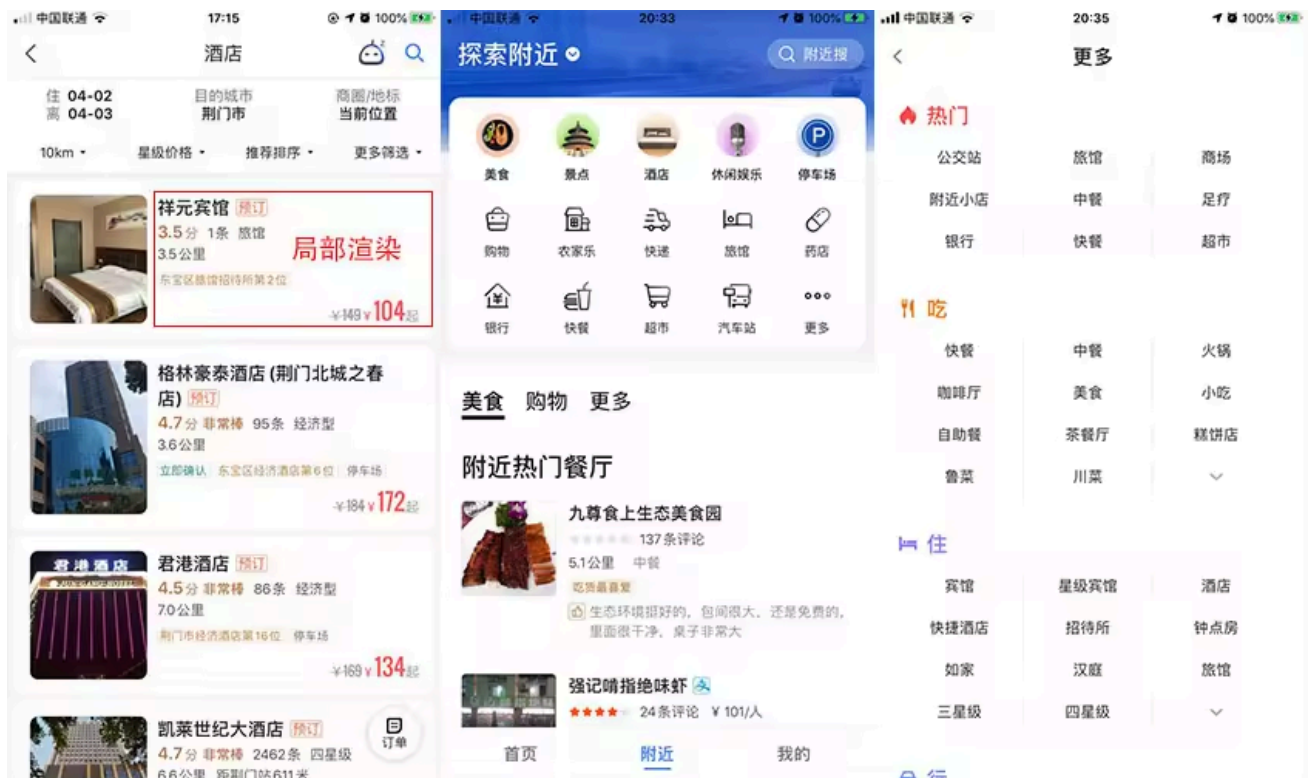
因为这些缺点AMAP只在三级页面及一些非核心业务的二级页面使用了这种跨端方式，这些只占AMAP业务很少的一部分代码，显然不能满足需求。



AJX 1.0 (XML Dom Engine)

随着业务的快速发展，之前一部分native实现的业务需要根据后台数据动态变更展示内容，当前的技术框架无法满足双端的这种需求，因此需要一种跨端的布局解析生成框架，XML dom engine，将后台配置的XML数据动态解析成各端的native组件，达到双端动态展示的目的，缺点：

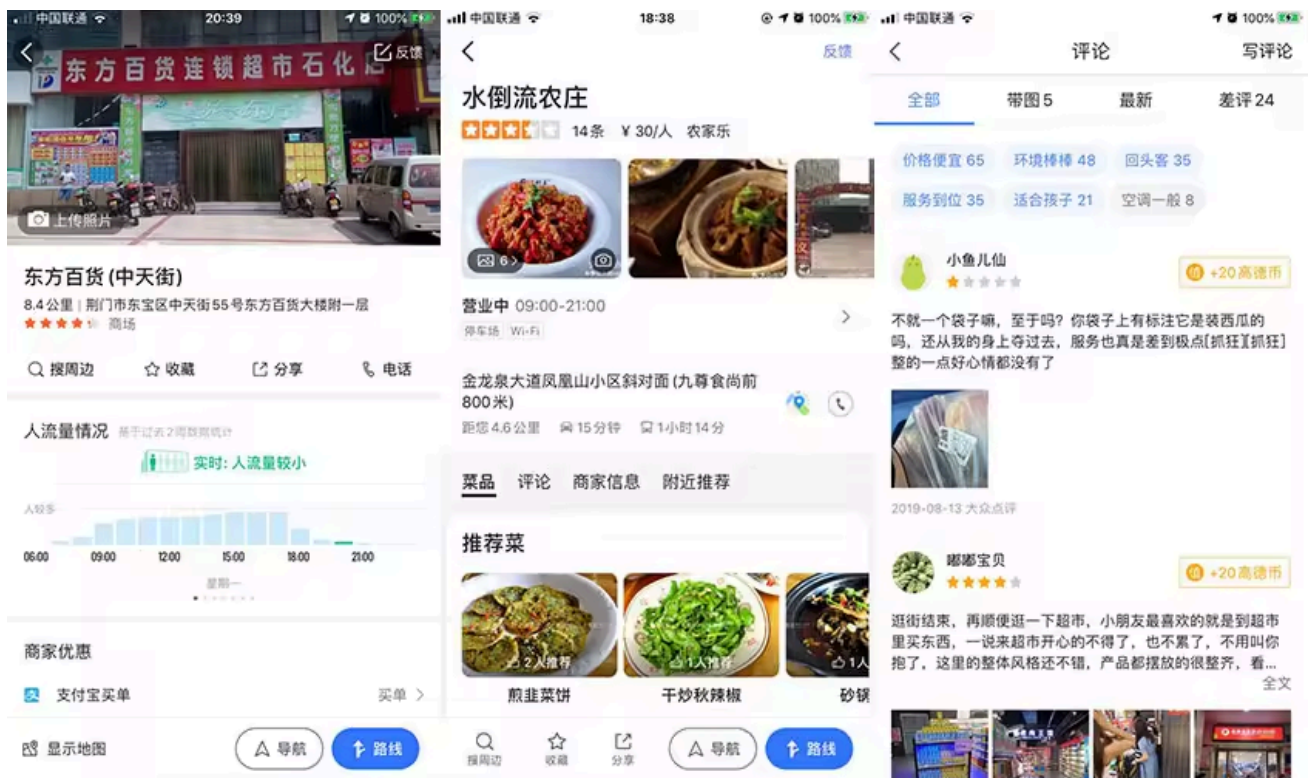
- 只能解析部分native组件
- 只支持简单样式的解析
- 不支持动态业务逻辑
- 只能用来做局部的动态更新



AJX 2.0

AJX 1.0 只达到了跨端UI展示，显然不能算是完整的跨端，而且只能实现简单的跨端UI展示需求。为实现完整跨端，在AJX 1.0基础上增加了JavaScript 引擎 (Duktape)支持动态业务逻辑、layout 渲染引擎支持更多的布局及样式渲染、native适配层支持更多原生组件，基本上算是实现了一个完整的跨端框架，能独立实现部分简单业务逻辑，缺点：

- Duktape虽然内存占用低，但是相对V8对JavaScript的解释执行速度上差了很多，只能实现简单业务逻辑
- js代码运行在主线程，有卡主线程的风险
- layout不是W3C标准设计，有大量偏向native布局的定制样式，使用上偏向native，无法跨端到H5
- 只支持简单的属性动画



AMAP跨端框架AJX 3.0

由于AJX 2.0在框架设计上存在的一些缺陷，在实际业务中并没有应用起来，为了解决AJX 2.0存在的缺陷需要对整个框架进行重构，重构版本定为3.0，当时业界比较流行的跨端框架是 react native（其它跨端框架还没有像现在这么火热），因此在重构AJX 3.0 时借鉴了

react native的部分框架设计，布局引擎直接切换成了Yoga(facebook开源项目)，对框架各层进行了细化拆分，整体上分为4层。

框架设计



- 引擎层：包含了JavaScriptCore/v8库，由C、C++实现的，唯一可以跟JS环境交互的功能模块，是连接JS与适配层之间的桥梁，也是跨端的基础
- 动态UI核心层，功能包括：解析JS和xml，生成DOM树，布局计算并通知适配层刷新UI
- 前端和适配层之间不能直接交互，都需要通过引擎层做转接
- 前端不感知UI线程，适配层不感知JS线程
- 适配层：基于iOS/Android平台的UI渲染层
- 渲染：根据引擎渲染接口指令，实现UI的渲染和事件分发
- 接口：根据引擎module接口，动态创建module对象和方法注册
- 布局：AJXEngine解析成绝对坐标，通过UI事件通知适配层刷新UI
- 业务层：按bundle(体现为不同ajx文件)划分为不同的业务模块，业务bundle间不可以依赖(代码、资源)，业务bundle可以依赖基础库bundle和基础组件bundle
- 动态化能力：分为热更新能力和Web化能力
- 热更新能力：主要用于修复线上业务bug或Crash，通过动态下载bundle文件的增量包并热加载得以实现；需要热更新的bundle必须在App内有对应的随版bundle.
- Web化：主要用于解决新业务在线上App上线的问题，即一个没有随版的bundle可以通过发布服务，在线上App上线并生效

框架优点

AJX 3.0 实现了一处编写多端（Android、iOS、H5）运行的目的，实现了真正意义上的跨端。

- 大大降低了客户端开发门槛，研发人员只要掌握前端开发的知识（html/css/Javascript）就能实现客户端业务的开发，而且前端流行的框架和库（react等）也可以低成本的移植过来，进一步提升业务的开发效率，AJX 3.0 类似一个定制的浏览器，跟传统浏览器的主要区别主要就在渲染层，AJX 会把UI渲染成各平台的原生组件，而浏览器有自己的渲染引擎，按照W3C标准绘制组件，用AJX开发能达到类似在浏览器上的开发体验。
- 业务研发人员不需要关注平台特性就能实现同样的效果，会由适配层统一处理成各平台标准组件，也不需要考虑兼容性问题，AJX 引擎层、适配层会统一处理
- AJX 开发的UI最终都会转化为native原生组件，因此在性能及使用体验上跟native非常接近
- 动态化能力，AJX开发的页面可以向网页一样实时加载展示，也可以像native一样打包发布，打包发布后的代码也能随时更新。不光解决了包大小问题，同时解决了APP更新发版慢问题，业务的迭代速度也快了很多，随时都能发布，不依赖客户端发版

```
import * as AJX from '@ajx_bundle_lib/FOXrax.js';
import FOXPage from '@ajx_bundle_lib/FOXPage.jsx';
import App from '@ajx_bundle_lib/FOXApp.js';

class HelloPage extends FOXPage {

  constructor(param) {
    super(param);
  }

  render() {
    return (<div class="page">
      <img src={require.resolve('../imgs/ajx.png')} />
      <label text="Hello AJX"/>
    </div>);
  }
}

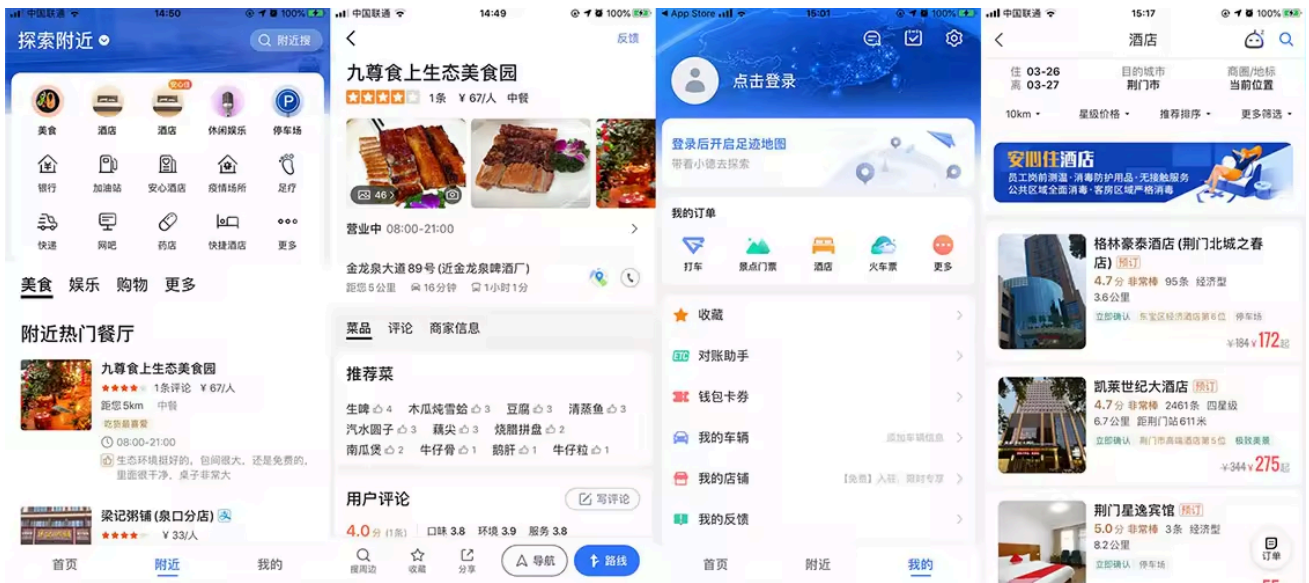
App.start(HelloPage);
```



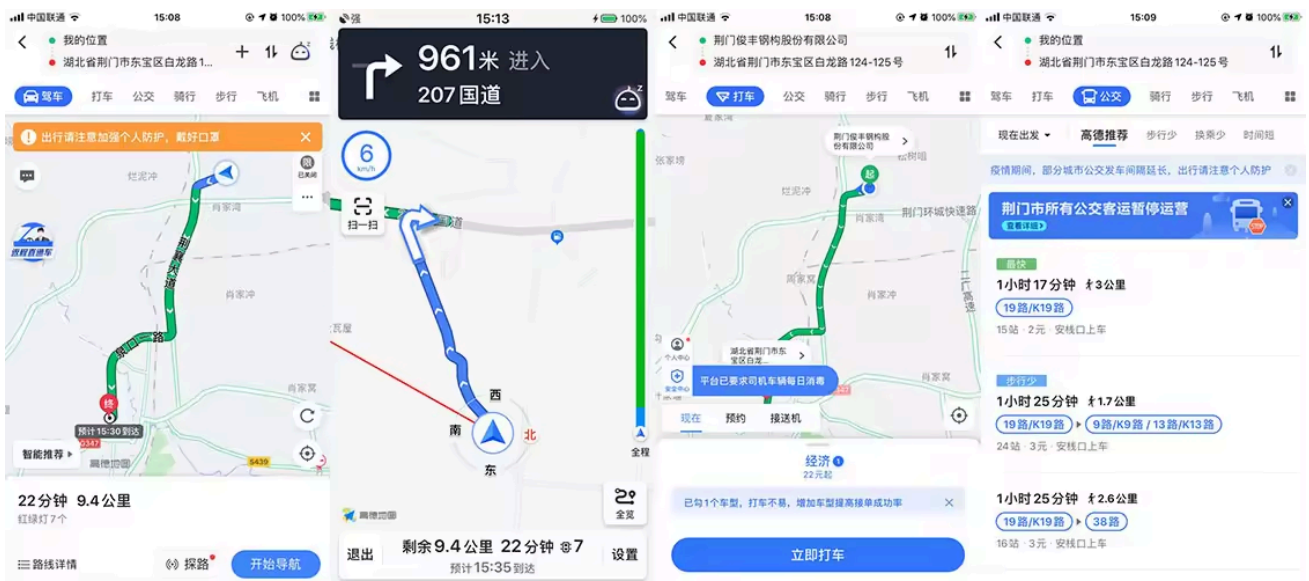
AMAP跨端业务应用

随着AJX3框架的完善，AJX3在AMAP所有业务上全面落地，2019年初已有近 60多个业务模块投入使用，约 60万行代码。AMAP常用业务基本上完成了AJX3的升级改造，大量的 native业务代码切换为AJX代码，包的大小也有所减小。之前开发这些业务的native研发人员都转到了AJX研发，之前是一个业务2个人做，现在是一个业务1人就可以做了，研发效率翻倍提升。

生活服务业务AJX应用



出行业务AJX应用



AMAP跨端框架问题

AJX3 固然能实现AMAP大部分业务需求，但是跟其它跨端框架一样也存在一些不好解决的问题。

- 动画：手势动画实现复杂，特别是页面级的，一个简单的关联动画可能就要几百行代码，效果可能还达不到要求，原因就是AJX多线程设计，JS运行在独立线程，但是用户操作在UI线程，因此无法在JS中根据手势实时操作UI，只能提前定义好手势动画规则，实现一

个完整动画往往需要多个规则相互关联来实现，造成复杂动画代码量大，代码阅读费劲维护成本高

- 组件：AJX UI 最终渲染成native组件，因为部分组件的平台特异性，很难保证它们的接口及展示保持统一，部分情况下需要做兼容处理，如表单、键盘等
- 性能：AJX 业务逻辑全是由JavaScript编写，JavaScript为解释执行类语言，本身性能就比Java、Object C差很多，在PC上由于硬件性能比手机高很多，很难感受出来，但手机上硬件性能比电脑差很多，代码量稍微大点就很明显，特别是对AMAP这类复杂应用，一个页面的代码很容易就能到万行，这个时候在代码加载时的耗时就很明显了

跨端框架未来发展

从AJX遇到的问题来看，依赖native渲染就会受到各平台内置组件的限制，而且还会有差异，想要避开这些限制只能跨端框架实现一套自己渲染引擎，在业务代码实现上需要考虑选用执行性能高的编译型语言，解释性语言在代码量大的业务上表现较差。目前比较火热的跨端框架flutter从设计上解决了上述问题，虽然才发布到1.9版本，但是在业界却异常火热，各大互联网厂商都在尝试，看来大家对flutter的期望很高，flutter最终是否能一统跨端江湖，我们拭目以待。